

Manual Completo do Ecossistema AIDD

Manual Completo do Ecossistema AIDD

Guia Prático End-to-End: Da Instalação ao Produção

Por: Ecossistema AIDD | Setembro 2026

Seção 1 — Visão Geral

O que é o Ecossistema AIDD?

O Ecossistema AIDD (AI-Driven Development) é um conjunto de 6 ferramentas integradas que transformam uma ideia em software testado e pronto para produção. Funciona como uma **fábrica de software completa**: cada ferramenta é uma estação de trabalho especializada, e todas seguem as mesmas regras de qualidade automáticas.

O que significa AIDD? Desenvolvimento Impulsionado por Inteligência Artificial. Na prática, significa que as ferramentas usam IA para acelerar o trabalho, mas sempre com verificações automáticas que garantem que o resultado está correto.

A grande diferença: O ecossistema funciona de forma idêntica independente de como você o usa — pelo terminal (CLI) ou conversando com um assistente de IA. A mesma governança, os mesmos testes, o mesmo resultado.

Novidades desta versão (15/09/2026): a stack de código gerado na Fase 8 foi atualizada (React 19 + FastAPI + PostgreSQL assíncrono, autenticação PyJWT + Argon2id, testes por propriedades, observabilidade com OpenTelemetry) e o Generator ganhou uma auditoria de **Tokenomics** que comprova em números reais a economia de tokens de IA. Detalhes na Seção 4.

As 6 Ferramentas

#	Ferramenta	O que faz	Comando	Quando usar
1	Forge	Instala regras de qualidade num projeto	<code>forge init</code>	Todo projeto novo ou existente
2	Generator	Cria um projeto completo a partir de uma ideia	<code>generate</code>	Quando você tem uma ideia e quer ver ela funcionando
3	Master	Adiciona novas funcionalidades a um projeto	<code>master add-module</code>	Quando seu projeto já existe e precisa crescer
4	Enterprise	Injeta componentes de segurança certificados	<code>enterprise inject</code>	Quando precisa de peças auditadas (auth, criptografia)
5	Ops	Configura servidores e faz deploy	<code>ops plan / ops deploy</code>	Quando o software está pronto para produção
6	Bridge	Migra apps de plataformas low-code para servidor próprio	<code>bridge scan</code>	Quando quer sair de Lovable/v0/Bolt

Os 16 Portões de Segurança (Quality Gates)

Todo código passa por 16 verificações automáticas antes de ser considerado "pronto". São como seguranças num aeroporto — nada passa sem ser verificado.

#	Nome	O que verifica
1	G_ECOSSISTEMA_INTEGRIDADE	Estrutura do projeto está correta
2	G_DRIFT_NUCLEO_COMPARTILHADO	Master e Enterprise não divergiram
3	G_HARNESS_COMPAT	Todos os assistentes de IA estão sincronizados
4	G_SEGREDOS	Não há senhas no código
5	G_CLI_HELP_CONSISTENCIA	Ajuda da CLI bate com as flags reais
6	G_COMPONENTE_AGNOSTICO	Componentes funcionam em qualquer lugar
7	G_ZERO_HEADLESS	Nenhum subagente invisível rodando
8	G_INFRA_COMPOSE	Docker Compose está correto
9	G_HADOLINT	Dockerfiles seguem boas práticas
10	G_TESTES_REAIS	Testes passam de verdade (sem simulação)
11	G_HONESTIDADE_ROTULO	Não há marketing exagerado nos gates
12	G_ARQUITETURA_DELIVERABLE	Código respeita Clean Architecture
13	G_DEPENDENCIAS_PIN_HASH	Dependências estão amarradas com hash
14	G_ESCRITOR_ATOMICO	Arquivos são escritos de forma atômica
15	G_TRANSACTION_LOG_LRU	Log de transações não está corrompido
16	G_UNIVERSAL_HARNESS	Compatibilidade com todos os harnesses

Validação em um comando: `python ecossistema.py audit` (exit 0 = tudo aprovado).

Gates extras por ferramenta: além destes 16 gates centrais, algumas ferramentas têm gates próprios — por exemplo, o Generator tem o G_TOKENOMICS, que audita se a economia de tokens de uma execução real bateu a meta (padrão: 30% mínimo). Esses gates extras somam uma camada de verificação específica da ferramenta; não substituem os 16 centrais (veja a Seção 4).

Seção 2 — Pré-requisitos e Instalação

O que você precisa ter instalado

Requisito	Versão mínima	Como verificar
Python	3.10+	<code>python --version</code>
Git	Qualquer	<code>git --version</code>
Terminal	Qualquer	PowerShell, CMD, Bash ou Zsh

Opcional mas recomendado:

- Um assistente de IA (Claude Code, Cursor, Antigravity, OpenCode, MimoCode etc.)

- Docker (para deploy em produção)

Instalação

Via terminal (Para todos)

```
# 1. Clonar o repositório
git clone https://github.com/heverton-dev/ecossistema-aidd.git
```

```
# 2. Entrar na pasta
cd ecossistema-aidd
```

```
# 3. Verificar se tudo está funcionando
python ecossistema.py status
```

Pronto. As dependências se instalam automaticamente na primeira execução (auto-bootstrap).

Se quiser forçar a instalação manualmente:

```
python ecossistema.py dependencia bootstrap
```

Via agente de IA (Para não-técnicos)

1. Abra o Claude Code, Cursor, Antigravity ou qualquer assistente de IA
2. Navegue até a pasta ecossistema-aidd
3. Digite no chat: `/dependencia bootstrap`
4. Pronto. O agente instala tudo automaticamente.

Como adicionar uma dependência nova depois:

Pelo terminal:

```
python ecossistema.py dependencia bootstrap
```

```
python ecossistema.py dependencia add-skill \
  --nome minhas-skill --pacote pacote-python \
  --instalar "pip install x" \
  --verificar "python -c 'import x'"
```

```
python ecossistema.py dependencia add-mcp \
  --nome meu-mcp --pacote pacote-python \
  --tipo stdio --comando "python mcp_server.py"
```

Pelo chat do agente:

```
/dependencia bootstrap
```

```
/dependencia skill minhas-skill
```

```
/dependencia mcp meu-mcp
```

Multi-Harness: Funciona em Qualquer Assistente

O ecossistema funciona com todos estes assistentes:

- **Claude Code** (Anthropic)
- **Cursor** (IDE com IA)

- **Antigravity** (agy)
- **MimoCode**
- **OpenCode**
- **Gemini CLI** (Google)
- **Hermes**
- **DeepSeek**

Quando você cria uma skill ou regra, ela é distribuída automaticamente para a pasta de cada assistente:

```
ecossistema-aidd/
├── .claude/           ← Regras para Claude Code
├── .cursor/          ← Regras para Cursor
├── .agent/           ← Regras para Antigravity
├── .mimocode/        ← Regras para MimoCode
├── .opencode/        ← Regras para OpenCode
├── .gemini/          ← Regras para Gemini CLI
└── .agents/          ← Regras para Hermes
```

Seção 3 — O Projeto Prático

O que vamos construir

Ao longo deste manual, vamos construir o **iTask** — uma plataforma SaaS de gestão de tarefas para equipes, do zero até produção.

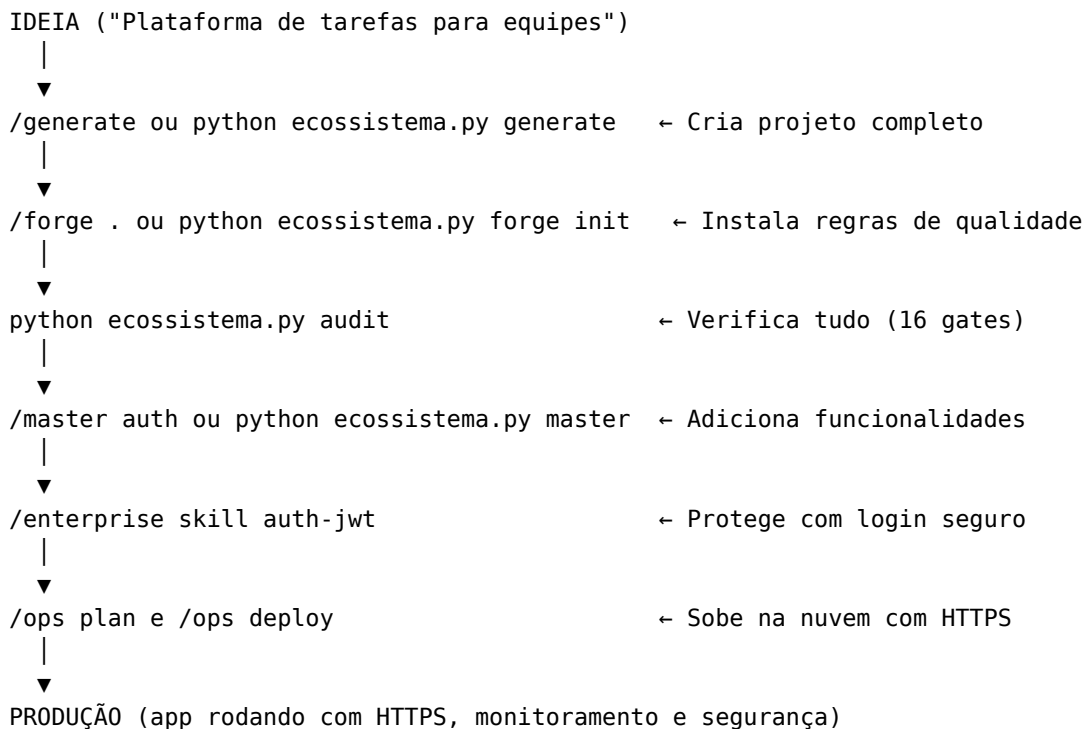
Funcionalidades do iTask

- Cadastro de usuários com autenticação segura (login/logout)
- CRUD completo de projetos (criar, listar, editar, excluir)
- Gerenciamento de tarefas com status (pendente, em andamento, concluída)
- Atribuição de tarefas entre membros da equipe
- Dashboard com visão geral dos projetos
- API REST documentada (OpenAPI/Swagger)
- Deploy em produção com HTTPS e monitoramento

O que cada ferramenta vai fazer no iTask

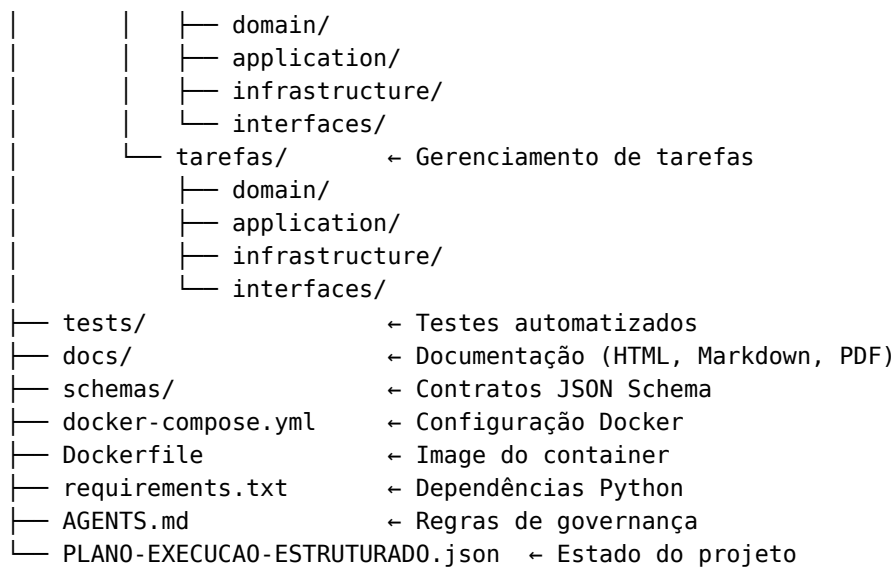
Etapa	Ferramenta	O que acontece
1	Generator	Cria o projeto completo (estrutura, schemas, docs)
2	Forge	Instala regras de qualidade e gates
3	Audit	Verifica se tudo está correto (16 gates)
4	Master	Adiciona módulos: auth, projetos, tarefas, dashboard
5	Enterprise	Injeta componente de autenticação JWT certificado
6	Ops	Configura servidor, Docker, HTTPS e monitoramento
7	Bridge	(Alternativa) Migra um app existente do Lovable

Fluxo resumido



Resultado esperado do projeto

```
itask/
├── src/
│   ├── modules/
│   │   ├── auth/ ← Autenticação (login, registro, JWT)
│   │   │   ├── domain/
│   │   │   ├── application/
│   │   │   ├── infrastructure/
│   │   │   └── interfaces/
│   │   └── projetos/ ← Gerenciamento de projetos
```



Seção 4 — Caminho 1: Via Terminal (CLI)

Para quem: Desenvolvedores, engenheiros de software, técnicos de TI.

O que você vai fazer: Digitar comandos no terminal para cada etapa.

Etapa 1: Criar o Projeto com o Generator

Objetivo: Transformar a ideia "plataforma de tarefas para equipes" numa estrutura de projeto completa com schemas, scripts, testes e documentação.

Criar o projeto (fases 1-7: estrutura + docs)

```
python ecossistema.py generate \
    "plataforma SaaS de gestão de tarefas para equipes\n  com autenticação, CRUD
de projetos, tarefas com\n  status e atribuição, dashboard e API REST" \
    --pasta ./itask
```

Criar com código funcional (fase 8: gera código via IA)

```
python ecossistema.py generate \
    "plataforma SaaS de gestão de tarefas para equipes\n  com autenticação, CRUD
de projetos, tarefas com\n  status e atribuição, dashboard e API REST" \
    --pasta ./itask --implementar-codigo
```

O que acontece: O Generator executa 8 fases automáticas:

Fase	Nome	O que faz	Depende de IA?
1	Pesquisa	Coleta informações sobre o domínio	Não
2	Análise	Decompõe o projeto em partes menores	Sim
3	Design	Decide a arquitetura (como as peças se encaixam)	Sim
4	Planejamento	Define tecnologias e estrutura	Não
5	Criação	Gera schemas, scripts e artefatos	Não
6	Documentação	Cria documentação completa (HTML, Markdown, PDF)	Não
7	Auto-crítica	Verifica se tudo está correto	Não
8	Implementação	(<i>opcional</i>) Gera código funcional via IA	Sim

Importante: As fases 1-7 são determinísticas — sempre funcionam igual. A fase 8 usa IA e pode gerar código que precisa de ajustes (taxa de sucesso de 55% a 91%).

Stack gerada pela Fase 8 (código funcional)

Desde a atualização de setembro/2026 (PLAN-0028), o código funcional da Fase 8 sai de fábrica com uma stack mais robusta, camada por camada:

Camada	Antes	Agora	Tecnologia
Frontend	4/10	8/10	React 19 + Vite + shadcn/ui (SPA completa)
Backend	5/10	8/10	FastAPI (OpenAPI 3.1 nativo, -46% código vs. versão anterior)
Autenticação	7/10	8/10	PyJWT + Argon2id, com fallback automático
Testes	7/10	9/10	Testes por propriedades com Hypothesis, além dos testes tradicionais
Observabilidade	7/10	9/10	structlog + OpenTelemetry
Banco de Dados	8/10	9/10	PostgreSQL assíncrono via asyncpg
Monitoramento	6/10	9/10	prometheus_client, com fallback próprio
Eventos	7/10	8/10	Worker assíncrono via Redis (BLPOP)
Nota média	6.5/10	8.5/10	—

Importante: cada peça "moderna" (PyJWT, Argon2id, asyncpg, prometheus_client) tem um caminho de fallback interno — se a dependência não estiver instalada, o sistema continua funcionando com uma implementação própria mais simples, sem travar por falta de lib externa. Isso segue o princípio de Universalidade do ecossistema.

Resultado esperado:


```

itask/
├── schemas/           ← Estruturas de dados (JSON Schema)
├── scripts/          ← Scripts Python
├── tests/             ← Testes automatizados
├── docs/              ← Documentação completa
└── PLANO-EXECUCAO-ESTRUTURADO.json ← Estado do projeto

```

Etapa 2: Instalar Governança com o Forge

Objetivo: Garantir que o projeto siga padrões de qualidade automáticos.

```

# Entrar na pasta do projeto
cd itask

# Instalar governança
python ecossistema.py forge init .

```

O que acontece:

1. AGENTS.md será criado na raiz — regras que qualquer assistente de IA entende
2. Verificações automáticas (gates) serão instaladas
3. O Git será configurado para manter consistência

Etapa 3: Rodar os Gates de Qualidade

Objetivo: Verificar se tudo está funcionando antes de continuar.

```
python ecossistema.py audit
```

Resultado esperado: Todos os 16 gates passam (exit 0).

Se algum gate falhar, o sistema diz exatamente o que está errado. Corrija e rode novamente.

Etapa 4: Verificar o Status do Ecossistema

```

# Ver status geral
python ecossistema.py status

# Rodar testes reais em cada ferramenta
python ecossistema.py status --testes

# Verificar binários do sistema (Git, Docker, Hadolint etc.)
python ecossistema.py preflight-host

```

Extra: Comprovando a Economia de Tokens (Tokenomics)

Para quem: mantenedores do aidd-generator que precisam provar, com números reais, que a estratégia de handoff entre fases economiza tokens de IA. Não faz parte do fluxo padrão de uso do ecossistema — é uma ferramenta de auditoria interna, sem comando equivalente em ecossistema.py ou slash command.

```

cd tools/aidd-generator

# Mede tokens reais (via tiktoken) do fluxo legado vs. otimizado

```

```
python scripts/benchmark_tokenomics.py --salvar benchmarks/benchmark.json

# Gera um dashboard HTML autocontido (gráfico de barras, custo em US$, assinatura
SHA-256)
python scripts/gerar_relatorio_tokenomics.py --gerar-e-abrir

# Audita uma execução real do pipeline contra o orçamento de tokens (padrão:
economia mínima de 30%)
python scripts/gates/G_TOKENOMICS.py
```

Resultado de uma execução real (rodada durante a escrita deste manual, ideia "app de hábitos"):

Fase	Tokens legado	Tokens otimizado	Economia
Handoff Fase 1→2	50.556	980	98,06%
Fase 8 (prompt + fix-loop)	12.235	2.214	81,90%
Total	62.791	3.194	94,91%

Isso equivale a **US\$ 0,2384 economizados** por execução (US\$ 0,2512 → US\$ 0,0128), com a medição assinada em SHA-256 para auditoria. O gate G_TOKENOMICS bloqueia (exit 1) execuções abaixo de 30% de economia ou que rotulem uma medição autodeclarada como "medição real".

Etapa 5: Adicionar Módulos com o Master

Objetivo: Construir as funcionalidades do iTask módulo por módulo.

5.1 — Módulo de Autenticação

```
python ecossistema.py master add-module auth
```

O Master cria uma "fatia vertical" completa:

```
src/modules/auth/
├── domain/           ← Regras de negócio (ex: "senha deve ter 8+ caracteres")
├── application/      ← Casos de uso (ex: "registrar usuário", "fazer login")
├── infrastructure/  ← Banco de dados (ex: tabela de usuários)
├── interfaces/      ← Rotas HTTP (ex: POST /auth/register, POST /auth/login)
├── models.py        ← Modelos de dados
├── services.py       ← Serviços (lógica principal)
└── routes.py        ← Rotas da API
```

5.2 — Módulo de Projetos

```
python ecossistema.py master add-module projetos
```

5.3 — Módulo de Tarefas

```
python ecossistema.py master add-module tarefas
```

5.4 — Módulo de Dashboard

```
python ecossistema.py master add-module dashboard
```

Cada módulo é independente — não quebra os outros. Comunicação entre módulos via EventBus, nunca import direto.

Etapa 6: Injetar Autenticação com o Enterprise

Objetivo: Proteger o sistema com login seguro usando componente certificado.

```
python ecossistema.py enterprise inject skill auth-jwt-service
```

O que isso faz:

1. O componente é verificado contra uma assinatura SHA-256 (impressão digital criptográfica)
2. Se o hash não bater (alguém alterou o componente), a injeção é bloqueada
3. Se bater, o componente é copiado para o seu projeto

Etapa 7: Sincronizar Componentes

Objetivo: Garantir que todos os assistentes de IA estejam com as mesmas regras.

```
# Sincronizar todos os componentes
```

```
python ecossistema.py components sync --tipo todos
```

```
# Verificar se está tudo sincronizado
```

```
python ecossistema.py components verify --tipo todos
```

Etapa 8: Configurar Infraestrutura com o Ops

Objetivo: Preparar o servidor para rodar o iTask em produção.

8.1 — Planejar a infraestrutura

```
python ecossistema.py ops plan \
```

```
"plataforma SaaS de tarefas para equipes\n  com autenticação, PostgreSQL,\n  Docker,\n  HTTPS e monitoramento"
```

8.2 — Testar o deploy (simulação segura)

```
python ecossistema.py ops deploy staging --dry-run
```

8.3 — Fazer o deploy real

```
python ecossistema.py ops deploy production
```

O que o Ops provisiona:

- **VPS** na nuvem (servidor virtual privado)
- **Docker** com cada parte do sistema num container isolado
- **Traefik** para roteamento (apontar domínio para o sistema certo)
- **PostgreSQL** para o banco de dados
- **HTTPS automático** (cadeado no navegador)
- **Uptime Kuma** para monitoramento (aviso se o servidor cair)
- **Hardening Ansible** (blindagem contra ataques)

8.4 – Verificar a infraestrutura do host

```
# Diagnóstico rápido dos binários do sistema
python ecossistema.py preflight-host

# Com JSON estruturado
python ecossistema.py preflight-host --json

# Auto-corrigir binários faltantes
python ecossistema.py preflight-host --fix
```

Etapa 9: Auditoria Final

Objetivo: Última verificação antes de considerar o projeto pronto.

```
# Rodar todos os 16 gates
python ecossistema.py audit

# Ver status completo com contagem de testes
python ecossistema.py status --testes
```

Resumo dos Comandos Usados (CLI)

Etapa	Comando	O que faz
1	python ecossistema.py generate "ideia" --pasta ./x	Cria projeto
2	python ecossistema.py forge init .	Instala governança
3	python ecossistema.py audit	Roda 16 gates
4	python ecossistema.py status	Ver status
5	python ecossistema.py master add-module x	Adiciona módulo
6	python ecossistema.py enterprise inject skill x	Injeta componente
7	python ecossistema.py components sync --tipo todos	Sincroniza
8	python ecossistema.py ops plan "desc"	Planeja infra
8	python ecossistema.py ops deploy staging --dry-run	Testa deploy
8	python ecossistema.py ops deploy production	Deploy real

Seção 5 – Caminho 2: Via Agentes de IA (Linguagem Natural)

Para quem: Não-técnicos, empreendedores, gestores, produtores de conteúdo.
O que você vai fazer: Digitar comandos em linguagem natural no chat de um assistente de IA.

Como funciona este caminho

Em vez de digitar comandos técnicos no terminal, você conversa com um assistente de IA (Claude Code, Cursor, Antigravity etc.) e descreve o que quer em linguagem normal. O assistente traduz suas palavras nos comandos corretos do ecossistema.

Exemplo:

- Você digita: `/generate plataforma de tarefas para equipes`
- O agente executa: `python ecossistema.py generate "plataforma de tarefas..." -- pasta ./itask`

Etapas 1: Criar o Projeto com o Generator

No chat do seu assistente de IA, digite:

```
/generate plataforma SaaS de gestão de tarefas para equipes  
com autenticação, CRUD de projetos, tarefas com status  
e atribuição, dashboard e API REST
```

O que acontece: O agente executa o Generator com suas instruções e cria o projeto completo — schemas, estrutura, testes e documentação — sem que você precise digitar nenhum comando técnico.

Etapas 2: Instalar Governança com o Forge

```
/forge .
```

O agente instala as regras de qualidade no seu projeto. É como contratar um inspetor de qualidade que verifica tudo automaticamente.

Etapas 3: Verificar se Tudo Está Correto

Verifique se o projeto está passando em todos os gates de qualidade

O agente roda `python ecossistema.py audit` e te mostra o resultado. Se algo falhar, ele te explica o que precisa corrigir.

Etapas 4: Adicionar Funcionalidades com o Master

Adicione cada funcionalidade do iTask uma por vez:

```
/master auth  
/master projetos  
/master tarefas  
/master dashboard
```

Cada comando cria uma parte completa do sistema — regras, banco de dados, API e tudo mais.

Etapas 5: Proteger com Segurança (Enterprise)

```
/enterprise skill auth-jwt-service
```

O agente injeta um componente de autenticação JWT que foi testado e certificado com assinatura digital SHA-256.

Etapa 6: Colocar no Ar (Ops)

`/ops plan "plataforma SaaS de tarefas para equipes\ncom autenticação, PostgreSQL, Docker, HTTPS e monitoramento"`

Depois de revisar o plano que o Ops gerou:

Faça o deploy em staging primeiro para testar

E quando estiver tudo certo:

Faça o deploy em produção

Etapa 7: Se o App Veio do Lovable (Bridge — Alternativa)

Se em vez de criar do zero, você já tem um app no Lovable:

`/bridge scan ./meu-app-lovable`

`/bridge convert-db ./meu-app-lovable`

`/bridge pack ./meu-app-lovable --domain tarefas.meudominio.com`

Depois, siga a partir da Etapa 2 (Forge) em diante.

Resumo dos Comandos Usados (Agentes)

Etapa	Comando no Chat	O que faz
1	<code>/generate <descrição></code>	Cria projeto
2	<code>/forge .</code>	Instala governança
3	<code>/audit</code> ou "verifique os gates"	Roda 16 gates
4	<code>/master <nome-do-módulo></code>	Adiciona funcionalidade
5	<code>/enterprise skill auth-jwt</code>	Injeta segurança
6	<code>/ops plan "descrição"</code>	Planeja infraestrutura
6	<code>/ops deploy</code>	Coloca no ar
7	<code>/bridge scan ./app</code>	Migra app low-code

Seção 6 — Guia Completo do Bridge (Migração de Apps Low-Code)

Para quem: Usuários que já têm um app no Lovable, v0 ou Bolt e querem sair da dependência da plataforma.

Quando usar o Bridge

- Você criou um app no Lovable e quer parar de pagar mensalidade
- Seu app está pronto e funcional, mas está "preso" na plataforma

- Você quer rodar o app no seu próprio servidor

Como usar (CLI)

1. Escanear o projeto (mapeia páginas, componentes, banco)

```
python ecossistema.py bridge scan ./meu-app-lovable
```

2. Converter o banco de dados (Supabase → PostgreSQL puro)

```
python ecossistema.py bridge convert-db ./meu-app-lovable
```

3. Migrar contas de usuários (senhas preservadas)

```
python ecossistema.py bridge migrate-auth \
  --source postgres://supabase-host:5432/postgres \
  --target postgres://minha-vps:5432/postgres
```

4. Empacotar com Docker + SSL

```
python ecossistema.py bridge pack \
  ./meu-app-lovable --domain tarefas.meudominio.com
```

5. (Opcional) Juntar múltiplos apps num só

```
python ecossistema.py bridge merge ./app1 ./app2 --output ./app-unificado
```

6. (Se precisar) Remover tudo da VPS

```
python ecossistema.py bridge destroy nome-do-app --domain tarefas.meudominio.com
```

Como usar (Agentes de IA)

```
/bridge scan ./meu-app-lovable
```

```
/bridge convert-db ./meu-app-lovable
```

```
/bridge pack ./meu-app-lovable --domain tarefas.meudominio.com
```

As 6 etapas do Bridge

Comando	O que faz	Quando usar
scan	Mapeia o projeto (páginas, componentes, banco)	Sempre, primeiro passo
convert-db	Converte banco Supabase para PostgreSQL puro	Quando usa Supabase
merge	Junta múltiplos apps num só	Quando tem vários apps
pack	Gera Docker + SSL pronto para deploy	Sempre, para empacotar
migrate-auth	Migra contas de usuários (senhas preservadas)	Quando tem login
destroy	Remove tudo da VPS (limpeza completa)	Quando quer recomeçar

Seção 7 — Referência Rápida

Todos os Comandos do Terminal

Comando	O que faz	Exemplo
status	Mostra estado do ecossistema	python ecossistema.py status
status --testes	Roda pytest em cada ferramenta	python ecossistema.py status --testes
audit	Roda os 16 gates	python ecossistema.py audit
preflight-host	Verifica binários do sistema	python ecossistema.py preflight-host
forge init	Instala governança	python ecossistema.py forge init .
forge inject	Injeta componente	python ecossistema.py forge inject skill x
generate	Cria projeto	python ecossistema.py generate "ideia" --pasta ./x
generate --impl-codigo	Cria projeto + código via IA	ecossistema.py generate "ideia" --pasta ./x
master add-module	Adiciona módulo	python ecossistema.py master add-module x
enterprise inject	Injeta componente certificado	python ecossistema.py enterprise inject skill x
ops plan	Planeja infraestrutura	python ecossistema.py ops plan "descrição"
ops deploy	Faz deploy	python ecossistema.py ops deploy staging --dry-run
bridge scan	Escaneia app low-code	python ecossistema.py bridge scan ./app
bridge convert-db	Converte banco	python ecossistema.py bridge convert-db ./app
bridge pack	Empacota Docker	python ecossistema.py bridge pack ./app --domain x
bridge merge	Junta apps	python ecossistema.py bridge merge ./a ./b --output ./c
bridge migrate-auth	Migra contas	python ecossistema.py bridge migrate-auth --source x --target y
bridge destroy	Remove da VPS	python ecossistema.py bridge destroy nome --domain x
dependencias	Lista dependências	python ecossistema.py dependencias nome-pacote tipo ytdio instalar z --verificar w

Todos os Slash Commands (Agentes de IA)

Comando	O que faz	Exemplo
/generate <descrição>	Cria projeto completo	/generate plataforma de tarefas
/forge .	Instala governança	/forge .
/master <módulo>	Adiciona funcionalidade	/master auth
/enterprise skill <nome>	Injeta componente	/enterprise skill auth-jwt
/ops plan "descrição"	Planeja infraestrutura	/ops plan "app com HTTPS"
/ops deploy	Faz deploy	/ops deploy
/bridge scan ./app	Escaneia app	/bridge scan ./meu-app
/bridge convert-db ./app	Converte banco	/bridge convert-db ./meu-app
/bridge pack ./app --domain x	Empacota Docker	/bridge pack ./app --domain x
/dependencia bootstrap	Instala dependências	/dependencia bootstrap
/dependencia skill <nome>	Adiciona skill	/dependencia skill minhas-skill
/dependencia mcp <nome>	Adiciona MCP	/dependencia mcp meu-mcp

Seção 8 — Glossário

Termo	Significado
CLI	Interface de linha de comando — usar o terminal em vez de botões
Slash Command	Comando no chat do assistente de IA (começa com /)
Quality Gate	Verificação automática que bloqueia código não-conforme
Vendor Lock-in	Ficar preso a uma plataforma difícil de sair
Clean Architecture	Organizar código em camadas (regras, dados, interface)
Fatia Vertical	Módulo completo e independente do sistema
Mock	Simulação falsa (proibido no ecossistema)
Result Monad	Padrão de "sucesso ou falha" em vez de quebrar o programa
SHA-256	Impressão digital criptográfica para verificar integridade
Zero-Trust	Não confiar em nada sem verificar primeiro
WAL	Write-Ahead Logging — modo seguro de escrever no banco SQLite
PostgreSQL	Banco de dados robusto e gratuito
Docker	Tecnologia de "containers" — isolar cada parte do sistema
HTTPS	Comunicação segura (o cadeado no navegador)
Traefik	Proxy reverso — roteia o tráfego para o serviço certo
VPS	Virtual Private Server — servidor seu na nuvem
Harness	Assistente de IA (Claude, Cursor, etc.)
Subagente	IA que trabalha em segundo plano numa tarefa específica
Context-Purge	Limpar o contexto da IA entre etapas (economizar tokens)
Tokens	Unidade de medida de texto para IAs (1 token ≈ 4 caracteres)
EventBus	Sistema de comunicação entre módulos (sem imports diretos)
JSON Schema	Contrato que define a estrutura dos dados
Tokenomics	Medição e comprovação de quanto uma estratégia economiza em tokens de IA
Argon2id	Algoritmo de hash de senha mais resistente que PBKDF2/bcrypt
OpenTelemetry	Padrão aberto para rastrear e observar sistemas distribuídos
Property-Based Testing	Testes que geram centenas de entradas aleatórias para validar uma regra geral (via Hypothesis), em vez de casos fixos
shadcn/ui	Componentes de interface prontos que você copia para o projeto (não é uma dependência de pacote)
AsyncPG	Driver assíncrono de PostgreSQL, usado para acessar o banco sem bloquear o servidor

Seção 9 — Solução de Problemas

Problema	Causa	Solução
ModuleNotFoundError	Dependências não instaladas	<code>python ecossistema.py --auto-bootstrap status</code> ou <code>python ecossistema.py dependencia bootstrap</code>
<code>exit 1</code> no audit	Gates falhando	Ler a mensagem de erro e corrigir o que o gate aponta
Generator cria estrutura vazia	Ideia muito vaga	Ser mais específico na descrição (ex: incluir funcionalidades, tipo de banco, etc.)
Master cria imports cruzados	Módulos acoplados	Comunicar via EventBus, nunca import direto
Bridge não encontra SQL	Estrutura diferente	Verificar se o projeto tem <code>supabase/migrations/</code>
Ops precisa de Linux	Ansible não roda no Windows	Usar WSL2 ou um container Docker
Tokens esgotando rápido	Contexto grande demais	Usar <code>context-purge</code> (o ecossistema faz isso automaticamente)
<code>preflight-host</code> falha	Binários faltando	<code>python ecossistema.py preflight-host --fix</code> para auto-corrigir
Harness com muitos dados	Cache crescendo	<code>python ecossistema.py harness clean</code>
Deploy não conecta na VPS	SSH não configurado	Verificar chave SSH e permissões no servidor

Estrutura do Repositório

```
ecossistema-aidd/
├── AGENTS.md           ← Regras fundamentais
├── ecossistema.py      ← CLI unificada
├── gates/              ← 16 Quality Gates
├── componentes/       ← Código compartilhado
│   ├── compartilhado/ ← Núcleo (Database, Result, EventBus)
│   └── aidd-master/   ← Componentes do Master
└── tools/              ← As 6 ferramentas
```

		aidd-forge/	← Governança
		aidd-generator/	← Fábrica
		aidd-master/	← Módulos
		aidd-enterprise/	← Segurança
		aidd-ops/	← Infraestrutura
		aidd-bridge/	← Low-code → VPS
		docs/	← Documentação
		MANUAL-COMPLETO.md	← Este manual
		ARQUITETURA-ECOSSISTEMA.md	
		CLI-REFERENCIA.md	
		MIGRACAO-PROJETO-EXISTENTE.md	
		CONTRIBUICAO.md	
		planos/	← Planos táticos e de auditoria
		protocolos/	← Protocolos canônicos
		relatorios/	← Relatórios formais
		prompts/	← Biblioteca de prompts
		features/	← Documentação de features
		explicacoes/	← Explicações detalhadas
		melhorias/	← Registro de melhorias
		testes/	← Baterias de testes E2E

Manual gerado em Setembro de 2026 pelo Ecossistema AIDD.

Última atualização: 15 de setembro de 2026 (stack pós-PLAN-0028 + auditoria de Tokenomics).

Distribuído sob a licença MIT.